



VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE307L_Compiler Design



Code Generation

Dr. B.V. Baiju,
SCOPE,
Assistant Professor
VIT, Vellore

Next-use information

- Next-use information is needed for *dead-code elimination* and *register allocation*.
- Next-use is computed by a **backward scan** of a **basic block**.
- The next-use information will **indicate the statement number** at which a particular variable that is defined in the current position will be reused.
- The "**use**" of a name in a three-address statement is defined as follows.
- Suppose three-address statement i assigns a value to x .
 - If statement j has x as an operand, and control can flow from statement i to j along a path that has no intervening assignments to x , then we say statement j uses the value of x computed at statement i .
 - We further say that x is live at statement i .

$i : x = b + c$

$j : y = x + b$

- Consider the given code snippet

```
x = y + z;  
a = x + b;  
c = x + d;
```

- In this code, the variable x is used three times,
 - The first time to store the result of $y + z$
 - The second time to store the result of $x + b$
 - The third time to store the result of $x + d$
- The *next use information for x* in this code would be the second and third uses.
- The compiler can use this information to optimize the code by **eliminating the load and store operations** for x between its uses.

```
t1 = y + z;  
a = t1 + b;  
c = t1 + d;
```

- In this optimized code, the variable x is no longer needed and has been replaced with a temporary variable t1.

- This optimization reduces the number of load and store operations and can improve the performance of the generated code.

Algorithm: Determining the liveness and next-use information for each statement in a basic block

INPUT : A **basic block B** of three-address statements. (We assume that the symbol table initially shows all non-temporary variables in B as being live on exit)

OUTPUT : At each statement **$i: x = y \text{ op } z$** in B , we attach to i the **liveness** and **next-use information** of x , y , and z

METHOD :

- We start at the last statement in B and scan backwards to the beginning of B .
- At each statement **$i: x = y \text{ op } z$** in B , we do the following:
 1. Attach to statement i the information currently found in the symbol table regarding the next use and liveness of x , y , and z .
 2. In the symbol table, set x to "not live" and "no next use."
 3. In symbol table, set y and z to "live" and the next uses of y and z to i .

- Consider the block

i : a = b + c

j: d = a + b

Non temporary variable will be considered as live.

- We have to start with **j: d = a + b** since it is backward algorithm . Refer 1 in algorithm.
- Initially all the variable are not having next use hence its null
- Left right variable (d) is set to "not live" and "no next use."
- Set *a* and *b* to "live" and the next uses of *a* and *b* to *j*
- Current symbol table information is applied to **i : a = b + c**
- Left right variable (*a*) is set to "not live" and "no next use."
- Set *b* and *c* to "live" and the next uses of *a* and *b* to *i*

	a	b	d
Live	True	True	True
Next use	None	None	None

	a	b	d
Live	True	True	False
Next use	j	j	None

	a	b	c
Live	True	True	True
Next use	j	j	None

	a	b	c
Live	False	True	True
Next use	None	i	i

- Consider the following basic block

1. $t1 = a * a$
2. $t2 = a * b$
3. $t3 = 2 * t2$
4. $t4 = t1 + t2$
5. $t5 = b * b$
6. $t6 = t4 + t5$
7. $x = t6$

Statements

7. No temporary is live
6. $t6$: use (7) $t4$, $t5$ not live
5. $t5$: use (6)
4: $t4$: use (6), $t1$ $t3$ not live
3. $t3$: use (4) $t2$ not live
2: $t2$: use (3)
1: $t1$: use (4)

Symbol Tables

$t1$ dead use in 4
 $t2$ dead use in 3
 $t3$ dead use in 4
 $t4$ dead use in 6
 $t5$ dead use in 6
 $t6$ dead use in 7

The 6 temporaries in the basic block can be packed into 2 locations $t1$ and $t2$

1. $t1 = a * a$
2. $t2 = a * b$
3. $t2 = 2 * t2$
4. $t1 = t1 + t2$
5. $t2 = b * b$
6. $t1 = t1 + t2$
7. $x = t1$

- **X is live at (5)**

- X is live at (5) because the value computed at (5) is used later in the basic block. X's next use at (5) is (14).
- It is a good idea to keep X in a register between (5) and (14).

```
(5)    X := ...
        ... (no ref to X) ...
(14)   ... := ... X ...
```

- **X is dead at (12)**

```
(12)   ... := ... X ...
        ... (no ref to X) ...
(25)   X := ...
```

- X is dead at (12) because its value has no further use in the block.
- Don't keep X in a register after (12).

Intermediate Code				Live/Dead				Next Use			
				x	y	z	t ₇	x	y	z	t ₇
(1)	x	:=	y+z	L	D	D		(2)	-	-	
(2)	z	:=	x*5	D		L		-		(3)	
(3)	t ₇	:=	z+1			L	L			(4)	(4)
(4)	y	:=	z-t ₇		L	L	D		(5)	(5)	-
(5)	x	:=	z+y	D	D	D		-	-	-	

x, y, z are live on exit